

Android Application Architecture

(part.02 – UI Layer)



목차

📖 Android App. Architecture 개요

📖 구현 준비

📖 UI Layer 구성

📖 FoodDBExam 전체 Architecture

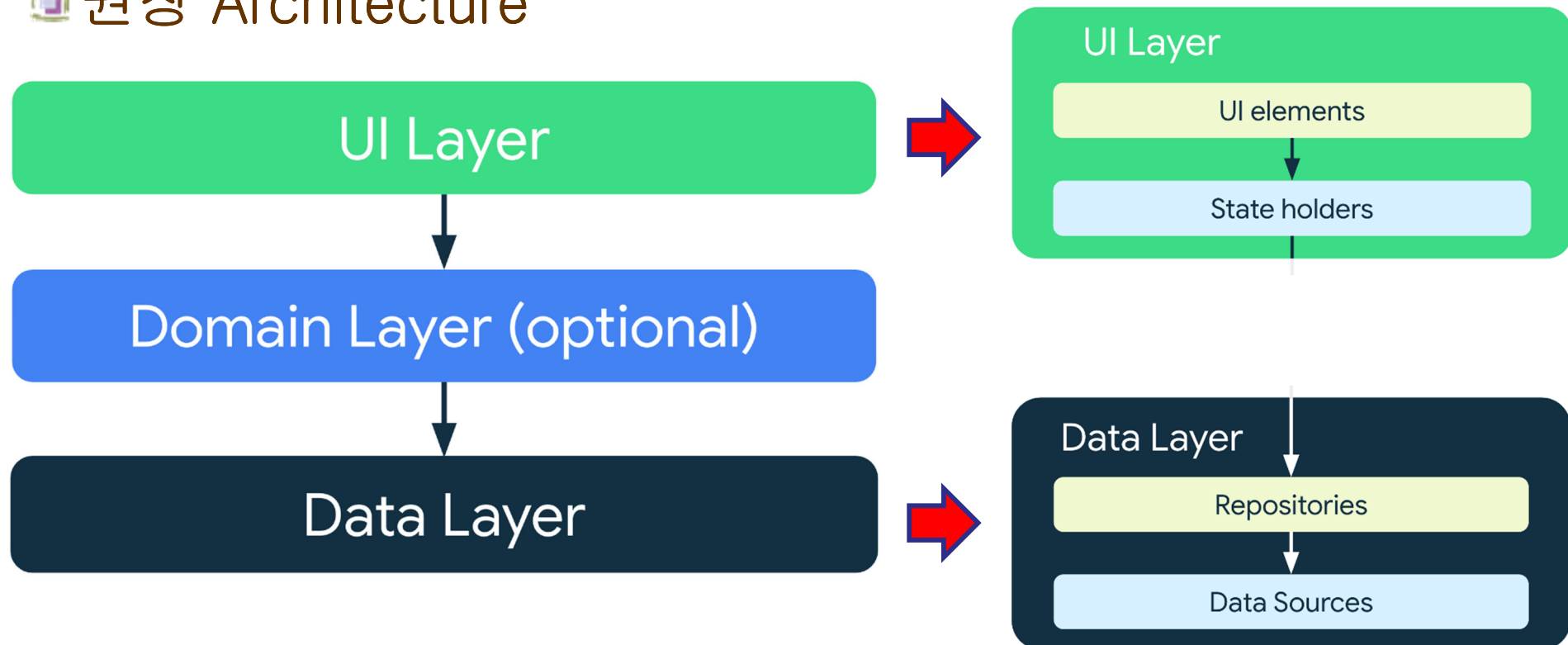
📖 참고사항

Android App. Architecture 개요

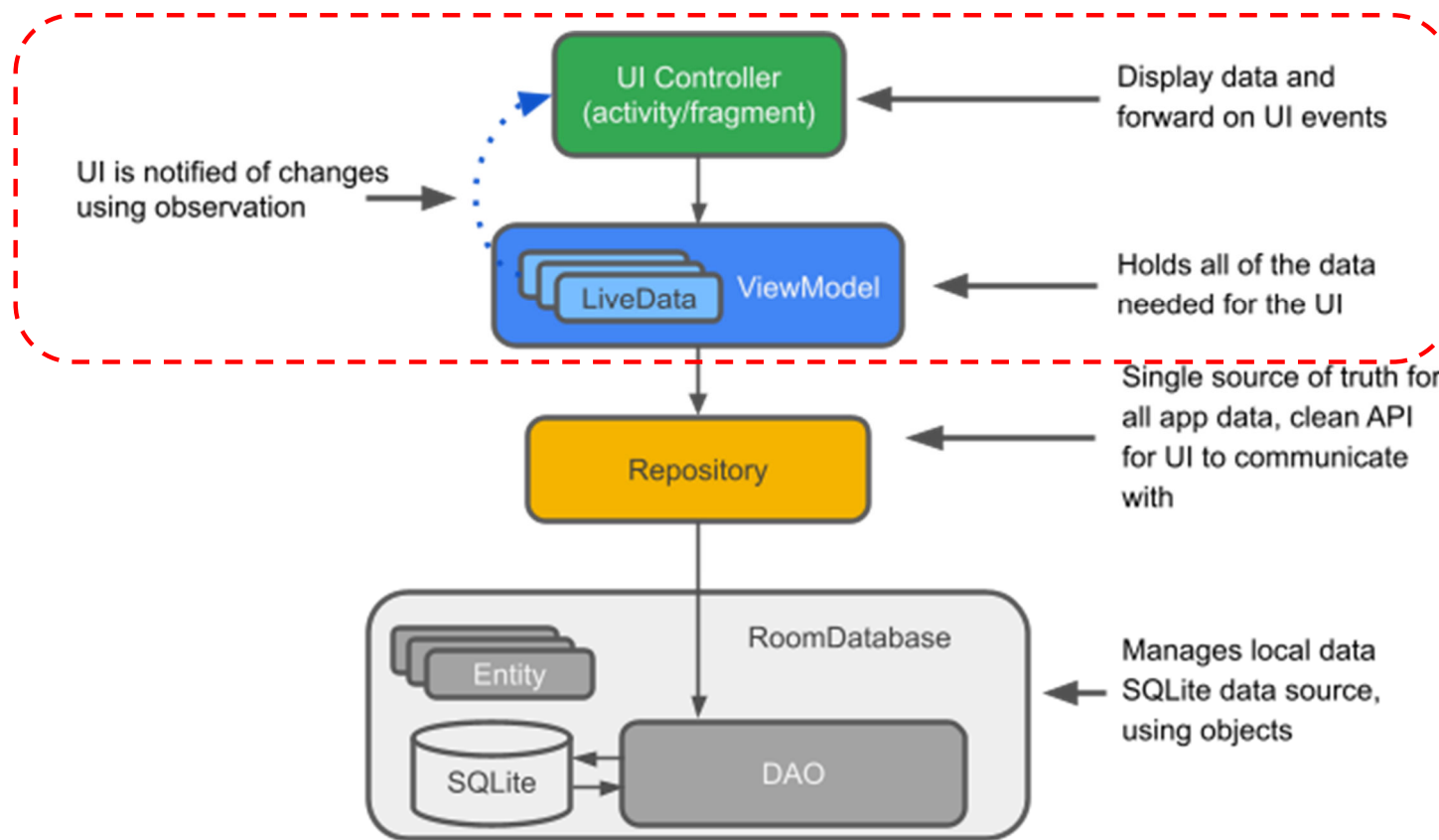
아키텍처 구성의 원칙

- ◆ 관심사의 분리(Separation of Concerns)
- ◆ 데이터 모델과 UI 의 분리
- ◆ Data의 단일 소스 저장소(SSOT: Single Source of Truth) 사용
- ◆ 단방향 데이터 흐름(UDF: Unidirectional Data Flow)

권장 Architecture



Architecture 주요 구성요소



- LiveData : 데이터의 변경을 관찰할 수 있는 데이터 보관 클래스
- ViewModel : Data 와 UI 사이의 통신을 담당 및 데이터 지속 보유
- Repository: 데이터 소스를 관리하는 개발자 구현 클래스
- Entity : ROOM 사용 시 DB 테이블 표현 및 데이터 보관
- RoomDatabase : 데이터베이스 관리, DAO 생성
- DAO : 데이터 접근 객체
- SQLite DB : 데이터 보관

출처: <https://developer.android.com/topic/architecture?hl=ko>



build.gradle(Project: ...)

```
plugins {  
    ...  
    id 'com.google.devtools.ksp' version '1.8.10-1.0.9' apply false  
}
```

Gradle Scripts
build.gradle (Project: AndroidArchitectureTest)

build.gradle(Module: ...)

```
plugins {  
    ...  
    id 'com.google.devtools.ksp'  
}  
  
dependencies {  
    ...  
    // ROOM  
    def room_version = "2.5.0"  
    implementation "androidx.room:room-runtime:$room_version"  
    annotationProcessor "androidx.room:room-compiler:$room_version"  
    implementation "androidx.room:room-ktx:$room_version"  
    ksp "androidx.room:room-compiler:$room_version"  
  
    // viewModels 사용  
    implementation "androidx.activity:activity-ktx:1.7.2"  
    // implementation "androidx.fragment:fragment-ktx:1.6.1"  
  
    // Lifecycle components  
    def lifecycle_version = "2.6.2"  
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version"  
    implementation "androidx.lifecycle:lifecycle-livedata-ktx:$lifecycle_version"  
    implementation "androidx.lifecycle:lifecycle-common-java8:$lifecycle_version"  
}
```

Gradle Scripts
build.gradle (Project: AndroidArchitectureTest)
build.gradle (Module :app)

ViewModel 등을 사용
하기 위해 적용

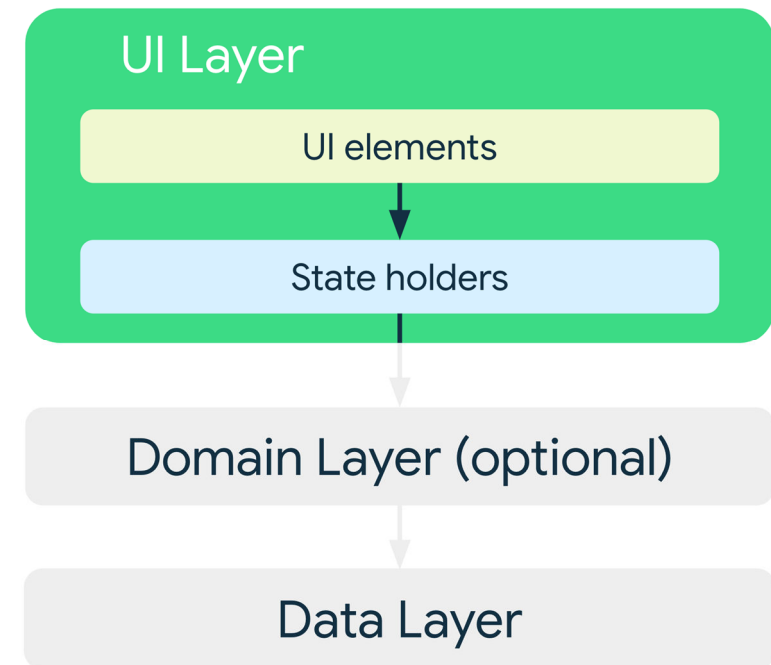
UI Layer

개요

- ◆ Presentation Layer 라고도 함
- ◆ 사용자의 상호작용을 처리하고, 데이터를 표시하는 역할
 - 데이터 레이어에서 가져온 데이터를 적절히 가공하여 UI에 표시

주요 구성 요소 in Android

- ◆ UI elements
 - 화면에 데이터를 렌더링하여 표시
- ◆ State holders
 - 데이터를 보유하고 UI에 표시, 필요 로직을 처리, UI의 데이터 상태를 보관

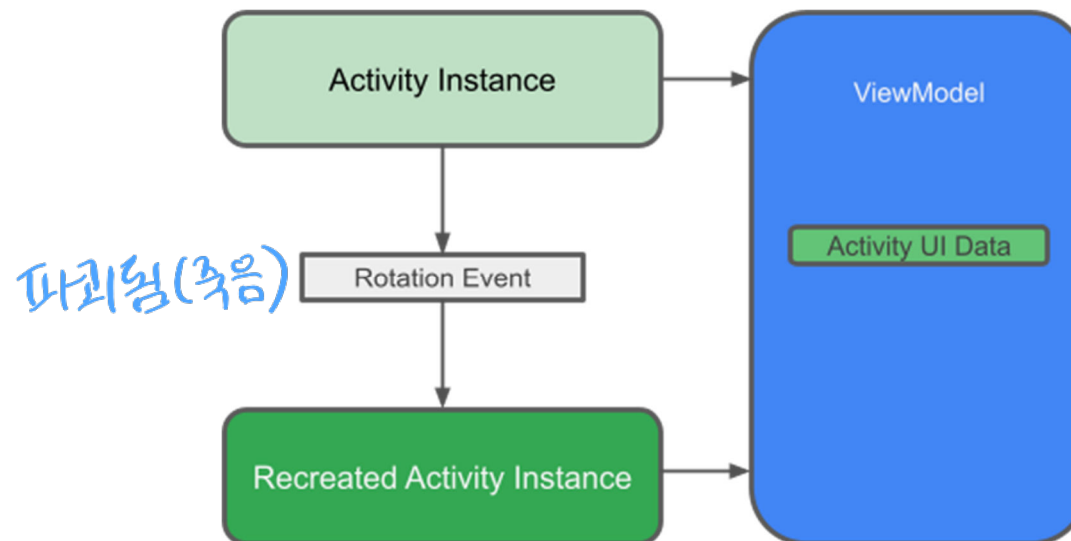


출처: <https://developer.android.com/topic/architecture/ui-layer?hl=ko>

UI layer State holder 의 구현

❏ ViewModel in Android

- ◆ UI state holder 역할을 수행하도록 구성된 클래스
 - Activity 등이 재생성 되어도 데이터 유지 가능
- ◆ Activity 또는 Fragment 등 화면 단위의 UI 데이터를 유지하고 처리
 - ≠ ■ MVVM 패턴의 ViewModel과의 차이점
 - Android Jetpack의 수명주기 구성요소에 포함
 - UI에 표시할 데이터를 보유하는 클래스 대체

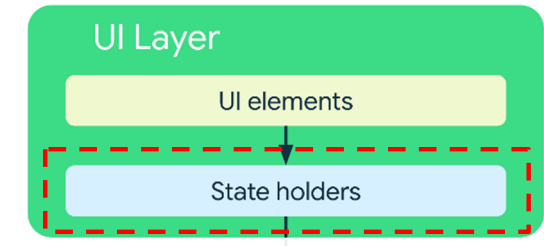


출처: <https://developer.android.com/codelabs/android-room-with-a-view-kotlin?hl=ko#9>

UI Layer 구성

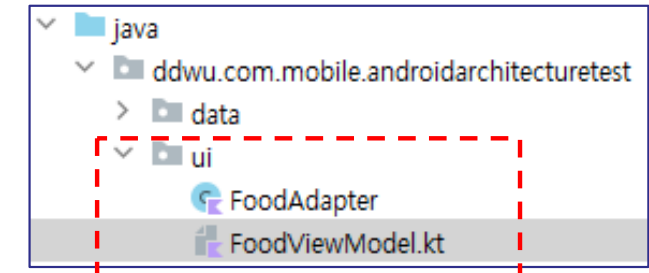
ViewModel의 추가

- ◆ ViewModel() 상속 및 구현



LiveData 사용

- ◆ 데이터 보관 클래스 → 변경 관찰
- ◆ Activity 등의 생명주기 인식 → UI에 적합



예: FoodViewModel

Flow<List<Food>> → LiveData<List<Food>> 로 변환

```
class FoodViewModel (var repository : FoodRepository) : ViewModel() {
    var allFoods: LiveData<List<Food>> = repository.allFoods.asLiveData()

    fun addFood(food: Food) = viewModelScope.launch {
        repository.addFood(food)
    }
}
```

코루틴 범위를 viewModelScope
범위로 지정

ViewModel 객체 생성



FoodViewModel를 위한 Factory 구현

- ◆ Factory 사용 → ViewModelProvider.Factory 상속 및 create(Class<T>) : T 재정의 → 생명주기 인식 자동 처리

repository 전달

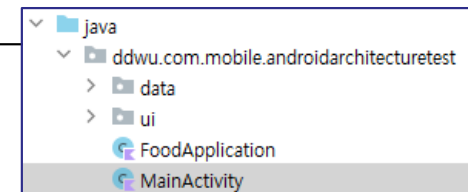
```
class FoodViewModelFactory(private val repository: FoodRepository) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(FoodViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return FoodViewModel(repository) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

FoodViewModel 객체 반환

객체 생성 : Activity의 변경 등에 상관 없이 유지

```
class MainActivity : AppCompatActivity() {
    val viewModel : FoodViewModel by viewModels {
        FoodViewModelFactory(application as FoodApplication).repository
    }
}
```

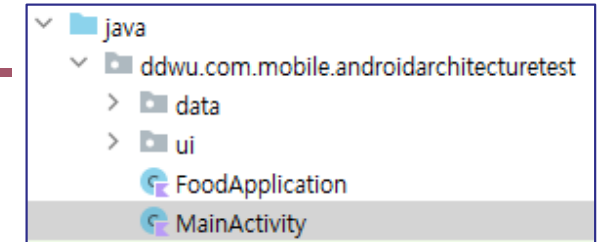
ViewModel delegate



application 멤버로 선언한 repository 사용

ViewModel을 사용한 데이터 접근

데이터 관찰 및 데이터 접근 수행



```
val viewModel : FoodViewModel by viewModels {
    FoodViewModelFactory( (application as FoodApplication).repository )
}

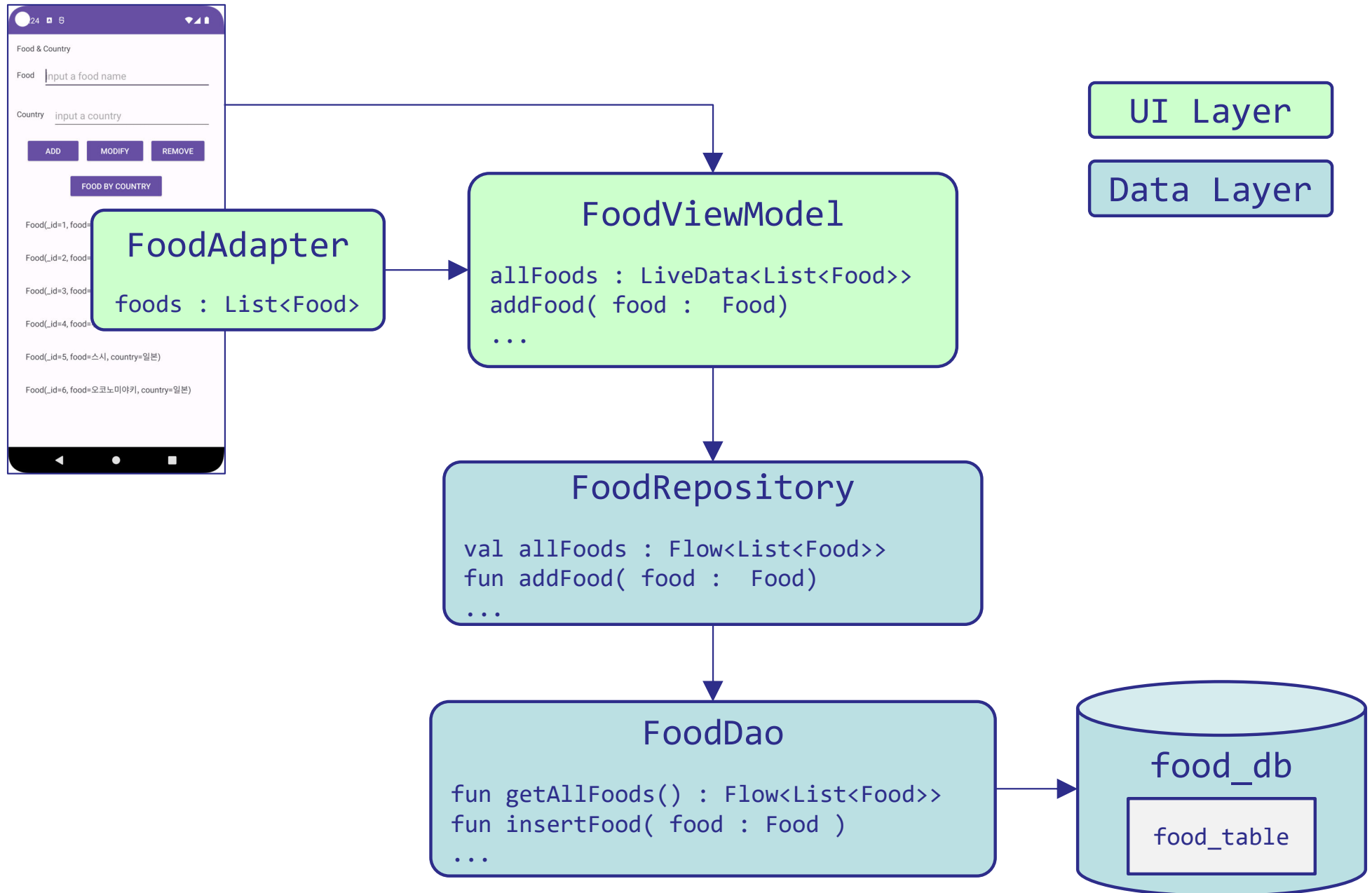
override fun onCreate(savedInstanceState: Bundle?) {
    ...
    viewModel.allFoods.observe(this, Observer {foods ->
        adapter.foods = foods
        adapter.notifyDataSetChanged()
        Log.d(TAG, "Observing!!!")
    })

    mainBinding.btnAdd.setOnClickListener {
        viewModel.addFood(
            Food(0, mainBinding.etFood.text.toString(),
                mainBinding.etCountry.text.toString())
        )
    }
}
```

LiveData.observe(context, Observer) 호출
// 데이터 변경을 지속적으로 관찰

ViewModel 에 정의한 함수를
호출하여 repository 접근

FoodDBExam 전체 Architecture



참고 사항

Coroutine 사용 시 반환 값 처리

◆ CoroutineScope.async { T } // ↔ launch

■ 코루틴 실행 후 Deferred<T> 반환

반환값(X)

◆ Deferred<T>

■ 코루틴 async 수행 결과를 저장하는 클래스

■ Deferred<T>.await() 으로 결과 전달 대기 후 T 반환

→ 결과를 받을 수 있다.

```
binding.btnShow.setOnClickListener {
    val country = binding.etCountry.text.toString()
    CoroutineScope(Dispatchers.IO).launch {
        val defFoods : Deferred<List<Food>> = foodViewModel.findFoodByCountry(country)
        val foods = defFoods.await()
        for (food in foods) {
            Log.d(TAG, food.toString())
        }
    }
}
```

결과 생성 시
까지 대기

①

②

```
fun findFoodByCountry (country: String) : Deferred<List<Food>> {
    val deferredFoods : Deferred<List<Food>> = viewModelScope.async {
        foodRepo.getFoodByCountry(country)
    }
    return deferredFoods
}
```

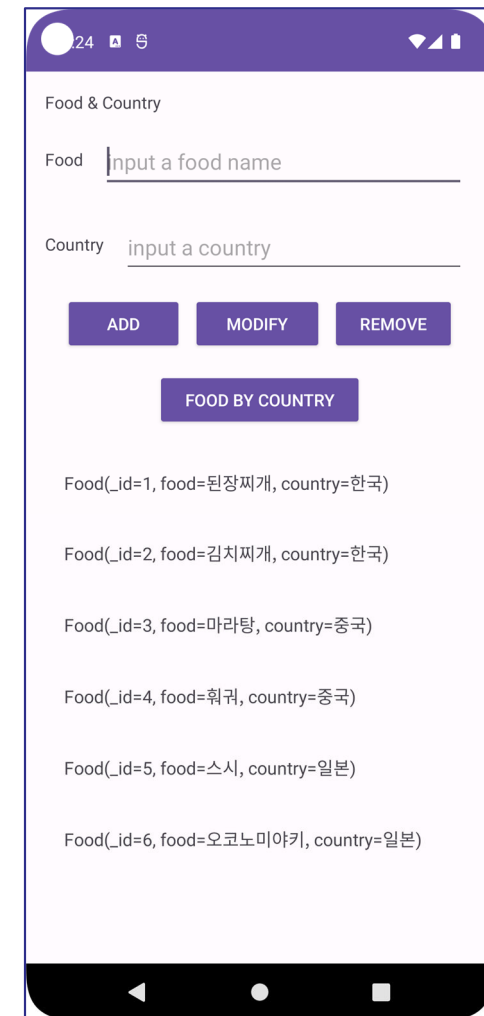
List<Food> 반환

Deferred<List<Food>> 반환

실습 1

다음 조건에 해당하는 앱을 구현하시오.

- ◆ FoodViewModel 구현
- ◆ 함수 추가 :필요에 따라 FoodDao 및 FoodRepository 수정
 - findFoodByCountry(country: String)
 - removeFoodByName(foodName: String)
 - 음식으로 삭제, repository 도 수정
 - modifyCountryByName(food: Food)
 - 음식으로 나라명 변경



참고

앱 아키텍처 가이드

- ◆ <https://developer.android.com/topic/architecture?hl=ko>

데이터 레이어 (Repository)

- ◆ <https://developer.android.com/topic/architecture/data-layer?hl=ko>

ViewModel 개요

- ◆ <https://developer.android.com/topic/libraries/architecture/viewmodel?hl=ko>

뷰를 사용한 Android Room – Kotlin

- ◆ <https://developer.android.com/codelabs/android-room-with-a-view-kotlin?hl=ko#0>